

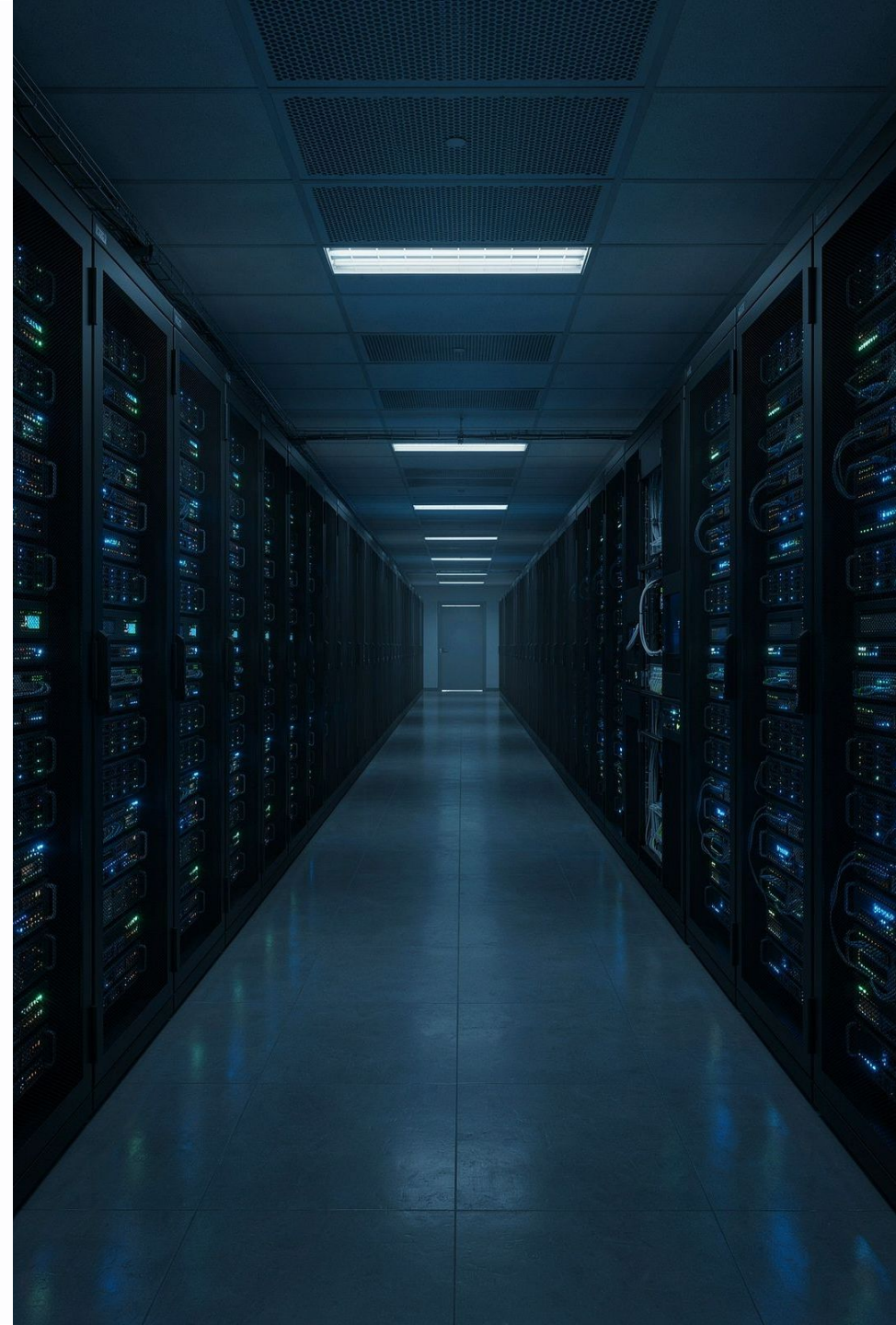
Day 3: Secure ML Workloads, Kubernetes & Risk Assessment

A practical, hands-on journey through containers, Kubernetes, machine learning operations, and security — built for DevOps, SRE, and ML engineering teams who need to ship secure, reliable ML workloads at scale.

7 MODULES

HANDS-ON

SECURITY-FIRST



What We'll Cover Today

Seven modules, one story – from packaging your first container to auditing a production ML cluster for risk. Each module builds on the last, so by the end you'll have a clear mental model of the entire secure ML delivery chain.

01

Containers

Packaging and image security

02

Kubernetes Fundamentals

Cluster architecture and workloads

03

Kubernetes Security

RBAC, policies, and secrets

04

ML Lifecycle

From raw data to deployed model

05

MLOps

Pipelines, CI/CD, and monitoring

06

Computer Vision

Real-world CV pipeline walkthrough

07

Risk Assessment

Audits, benchmarks, compliance

MODULE 1

Containers: Packing Your App Like a Shipping Container

Before Kubernetes, before ML pipelines — everything starts with a container. Think of it like a shipping container at a port: it doesn't matter if you're shipping toys or electronics, the container is a standard box that any crane (any host) can handle.



Container Fundamentals & Docker Architecture

A container bundles your application code, runtime, libraries, and config into one portable unit. Docker popularized this model with a simple client-server architecture: the **Docker CLI** talks to the **Docker Daemon**, which builds images, runs containers, and pulls from registries.

Real-World Analogy

Imagine a chef (your app) who travels with their own kitchen toolkit (the container). No matter which restaurant (server) they work in, they have exactly what they need — no surprises.

Key Docker Components

- **Image** — the blueprint (read-only)
- **Container** — a running instance of an image
- **Registry** — a warehouse storing images (e.g. Docker Hub, ECR)
- **Daemon** — the engine that manages everything

Image Security & Scanning: Don't Ship Vulnerabilities

An image is only as trustworthy as what's inside it. A poorly chosen base image can pack in hundreds of known vulnerabilities before you write a single line of your own code. Image scanning tools like **Trivy**, **Snyk**, or **Grype** inspect every layer and flag CVEs before the image ever reaches production.

Use Minimal Base Images

Prefer `distroless` or `alpine` images. Fewer packages = fewer attack vectors. A full Ubuntu base image can carry 400+ packages you never use.

Scan Early, Scan Often

Integrate image scanning into your CI pipeline. Block images with CRITICAL or HIGH severity CVEs from being pushed to your registry.

Sign Your Images

Use **Cosign** or **Notary** to cryptographically sign images. This ensures that what you built is exactly what gets deployed — no tampering in transit.



Never run containers as root. Add `USER nonroot` in your Dockerfile to drop privileges before the app starts.



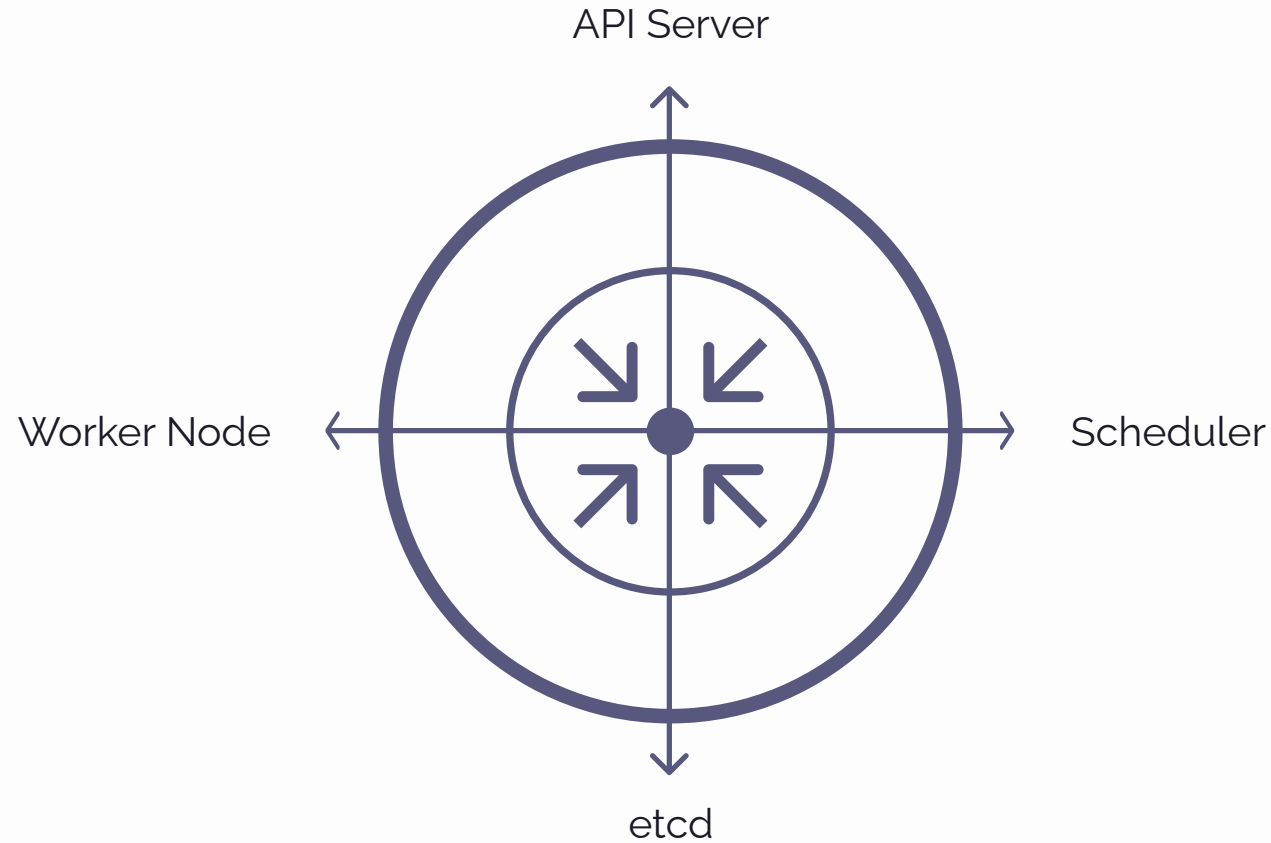
MODULE 2

Kubernetes Fundamentals: The Control Tower for Your Containers

If containers are the planes, Kubernetes is air traffic control. It schedules, scales, heals, and routes traffic across a fleet of nodes — automatically. Understanding its architecture is the foundation for everything that follows.

Cluster Architecture: Control Plane & Workers

A Kubernetes cluster has two tiers: the **Control Plane** (the brain) and **Worker Nodes** (the muscles). The control plane makes decisions; the workers do the actual work of running your containers.



Every action you take with `kubectl` goes through the API Server – the single front door to the entire cluster.

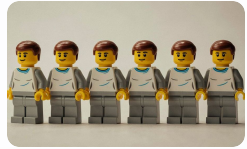
Core Kubernetes Objects Explained Simply

Kubernetes uses a set of building blocks to describe what you want to run and how. Here's how they relate to each other in a real ML inference service:



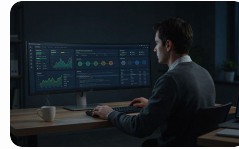
Pod

The smallest unit — one or more containers sharing a network. Like a single worker at a desk.



ReplicaSet

Keeps N copies of your Pod alive at all times. If one crashes, it spins up a replacement automatically.



Deployment

Wraps a ReplicaSet and adds rolling updates, rollbacks, and version history. Use this for stateless services.



Service

A stable network address in front of your Pods. Routes traffic even as Pods come and go.

ConfigMaps, Secrets & Ingress

Three more objects that every ML workload relies on. Together they handle configuration, credentials, and external traffic routing — keeping these concerns cleanly separated from your application code.

ConfigMap

Non-sensitive config values
— model paths, feature flags,
API endpoints. Decouples
config from code.

Secret

Base64-encoded sensitive
data — API keys, DB
passwords. Stored
separately, injected at
runtime.

Ingress

The front door for HTTP(S) traffic into your cluster. Routes
external requests to the right Service by hostname or path.

Real-World Example

Your model serving API reads its model version from a **ConfigMap**, its database password from a **Secret**, and is exposed to the internet through an **Ingress** with TLS termination. Change any of these without redeploying your app — that's the power of separation of concerns.

MODULE 3

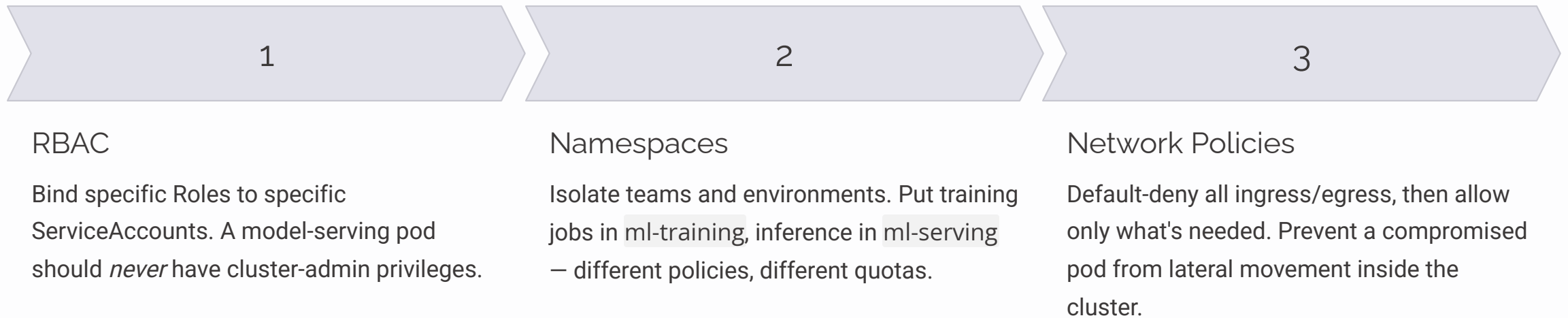
Kubernetes Security: Locking Down the Cluster

A misconfigured Kubernetes cluster is one of the most common sources of cloud security incidents. Module 3 walks through the key controls that turn a default, open cluster into a hardened, production-grade environment.



RBAC, Namespaces & Network Policies

The first three layers of Kubernetes security answer three questions: **who** can act (*RBAC*), **where** can they act (*Namespaces*), and **who can talk to whom** (*Network Policies*).



📌 Apply the principle of least privilege everywhere: least access, least network, least capability.

Pod Security, Secrets Management & Admission Controllers

Even with RBAC in place, a pod can still do damage if it runs as root, mounts the host filesystem, or reads secrets it shouldn't. Pod Security Standards and proper secret management close these gaps.

Pod Security Standards

Kubernetes defines three levels — **Privileged**, **Baseline**, and **Restricted**. For ML workloads, enforce **Restricted**: no root, no privilege escalation, read-only root filesystem where possible.

Secure Image Practices

Only pull from approved private registries. Use image digests (SHA256) instead of mutable tags like :latest.

Secrets Management

Kubernetes Secrets are base64-encoded — not encrypted by default. Use **Vault** (HashiCorp) or **AWS Secrets Manager** with the Secrets Store CSI Driver to inject secrets at runtime without storing them in etcd in plaintext.

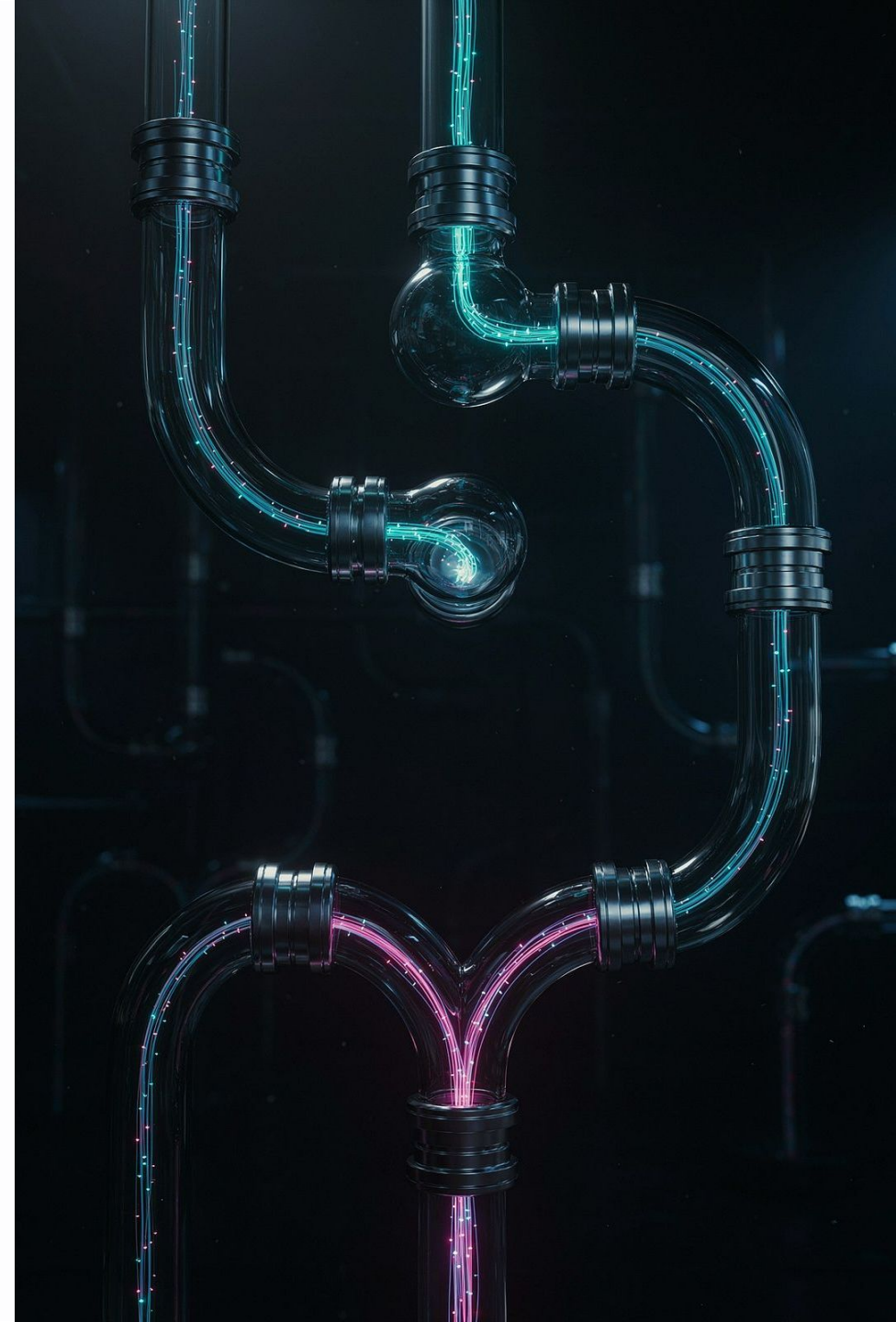
Admission Controllers

Webhooks that intercept every API request before it's persisted. Tools like **OPA Gatekeeper** or **Kyverno** enforce policy as code — e.g., block any image not from your approved registry.

MODULE 4

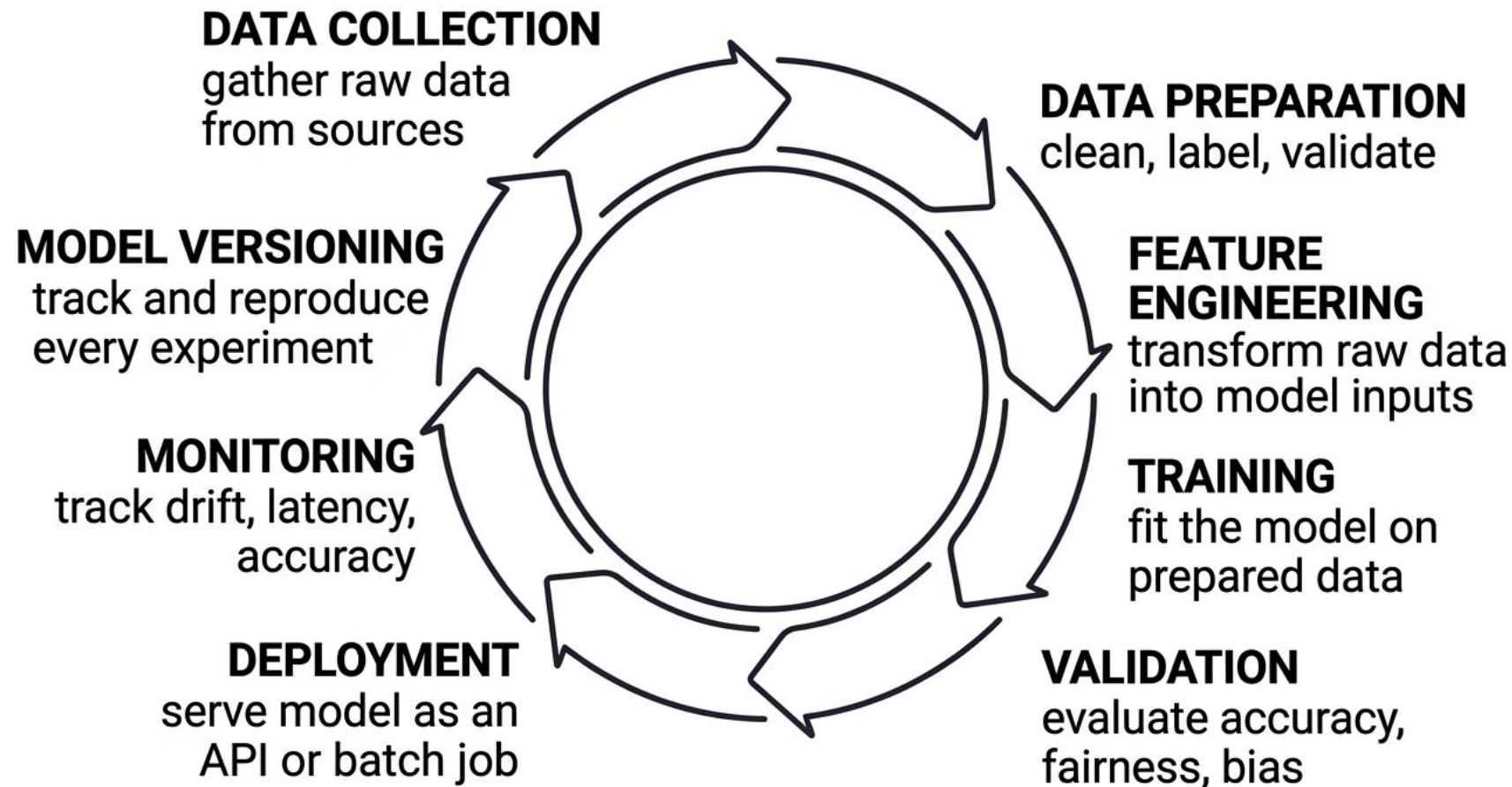
The ML Lifecycle: From Raw Data to Deployed Model

Building an ML model isn't a single step — it's an eight-stage journey. Each stage introduces unique security, quality, and reproducibility concerns that DevOps and ML engineers must address together.



The 8 Stages of the ML Lifecycle

Think of training a fraud detection model: you collect transactions, clean the data, engineer features like "transactions per hour," train a classifier, validate its precision, deploy it as an API, monitor drift, and version every model artifact. This is the full loop.



MODULE 5

MLOps: Bringing DevOps Discipline to Machine Learning

MLOps applies the proven practices of software DevOps — automation, versioning, continuous delivery — to the unique challenges of ML: data drift, model decay, experiment reproducibility, and large artifact management.



MLOps Core Practices

A mature MLOps platform automates the path from a data scientist's experiment to a production-grade, monitored model. Here's how each piece fits together in a real organization:



ML Pipelines

Orchestrate data prep → training → evaluation as code using tools like **Kubeflow Pipelines** or **Argo Workflows**. Every run is reproducible and auditable.



CI/CD for ML

Automatically retrain and redeploy models when new data arrives or code changes. Gate deployments on validation thresholds — no human needed for routine updates.



Model Registry

A central catalog of every trained model with metadata: accuracy, training data version, who approved it, and its current deployment status. Think of it as GitHub for models.



Experiment Tracking

Tools like **MLflow** or **Weights & Biases** log every hyperparameter, metric, and artifact so you can reproduce any result from six months ago.



Monitoring Concepts

Track model performance in production: data drift (inputs shifting), concept drift (the world changing), latency, and error rates. Alert before accuracy degrades visibly.



REST API Inference

Serve predictions over HTTP using frameworks like **TorchServe**, **Triton Inference Server**, or a simple FastAPI wrapper — secured behind Kubernetes Ingress with mTLS.



MODULE 6

Computer Vision: Teaching Machines to See

Computer vision (CV) is one of the most widely deployed ML domains — from defect detection on factory floors to real-time object detection in autonomous vehicles. Understanding the pipeline helps you deploy and secure it correctly.

The Computer Vision Pipeline End-to-End

A real-world CV system — say, detecting hard-hat compliance on a construction site — flows through four stages. Each stage has distinct infrastructure and security requirements on Kubernetes.



Image Classification

Assigns a single label to the whole image — "hard hat: yes/no." Models: ResNet, EfficientNet. Simpler, lower latency.

Object Detection

Locates and labels multiple objects within an image with bounding boxes — "3 workers, 2 without hard hats, location: upper left." Models: YOLO, Faster R-CNN.



MODULE 7

Security Audits & Risk Assessment: Know Your Exposure

You can't secure what you don't understand. Module 7 gives you a structured methodology to identify risks, quantify vulnerabilities, and measure your Kubernetes and cloud environment against industry-standard benchmarks — before an attacker does it for you.

Risk Assessment Framework for ML on Kubernetes

A rigorous security audit follows a repeatable process. For ML workloads, common findings include over-permissioned service accounts, unencrypted model artifacts in object storage, and publicly accessible inference endpoints — all addressable with the controls we covered today.

1

Risk Identification

Map all assets: clusters, registries, data stores, model endpoints. Ask: what's exposed, who has access, what's the blast radius of a breach?

2

Vulnerability Assessment

Run automated scanners (Trivy, kube-bench) against images and cluster configs. Prioritize CRITICAL and HIGH findings for immediate remediation.

3

Security Audits

Periodic manual reviews of RBAC bindings, network policies, and secret hygiene. Automated tools miss logic errors that a trained eye catches.

4

CIS Benchmarks

The CIS Kubernetes Benchmark provides 100+ controls covering API server flags, etcd encryption, kubelet config, and more. Use **kube-bench** to score your cluster automatically.

5

Cloud Misconfigurations

Public S3 buckets, open security groups, unused IAM roles with broad permissions — cloud misconfig is the #1 cause of data breaches. Use tools like **Prowler** or **ScoutSuite**.

6

Compliance

Map your controls to frameworks like SOC 2, HIPAA, or ISO 27001. Document evidence continuously — don't scramble at audit time.



Day 3 Wrap-Up: The Full Picture

Today you connected seven critical domains into one coherent story — from a single Dockerfile to a fully audited, production ML cluster. Security isn't a layer you add at the end; it's a discipline woven through every stage of the pipeline.

Build Securely

Minimal images, scanning, signing — secure the artifact before it ever reaches the cluster.

Deploy Safely

RBAC, Network Policies, Pod Security Standards, and Admission Controllers limit the blast radius of any incident.

Operate Continuously

MLOps pipelines, monitoring, and model versioning keep production reliable and reproducible over time.

Audit Relentlessly

Regular risk assessments and benchmark scans ensure your posture doesn't drift as your platform evolves.